

TimeHAT NIC Integration Requirements and Phased Implementation Specification

Post-FE-5680A Integration Architecture for Raspberry Pi 5

Purpose: Define a post-FE-5680A TimeHAT NIC integration plan for Raspberry Pi 5 that adds NIC-based timestamping, PPS I/O, and later PTP service capability without changing the existing ZED/FE/Teensy truth hierarchy.

Document type	Requirements and phased implementation specification
Project phase	After FE-5680A integration is completed
Assumed baseline	ZED truth + FE disciplined flywheel + Teensy measurement engine
Core architectural constraint	TimeHAT is an interface and service layer, not a default truth replacement

1. Document purpose

This document defines the project goals, architecture guardrails, functional requirements, non-functional requirements, interface expectations, telemetry needs, dashboard implications, verification plan, and phased implementation strategy for integrating a TimeHAT NIC on a Raspberry Pi 5 ****after FE-5680A integration is completed****.

The intent is to extend the system with:

- PCIe Intel i226 network hardware timestamping on Raspberry Pi 5
- PPS input/output capabilities through the TimeHAT timing interfaces
- later PTP-capable network timing services
- optional TimeHAT-assisted NTP/PTP server roles

This specification is written so that TimeHAT integration does ****not**** collapse the existing timing truth model already established for the ZED-F9T, FE-5680A, Teensy, Raspberry Pi, and Piksi.

2. System context

2.1 Existing post-FE target system

This specification assumes the FE-5680A project has already been completed to the point that:

- ZED-F9T remains the sole GNSS truth source
- ZED PPS remains the primary PPS truth source
- Teensy remains the measurement and timing analytics engine
- FE-5680A functions as the disciplined local oscillator and holdover flywheel
- Piksi remains a PPS comparison reference only
- Raspberry Pi continues to host the database, dashboard, and time services
- the dashboard and database distinguish corrected timing truth from raw offsets

2.2 Planned TimeHAT role

The TimeHAT NIC shall be added as a ****network timing interface and timestamping platform**** for Raspberry Pi 5, not as a replacement for the existing truth hierarchy.

Primary intended functions:

- provide an Intel i226 NIC on the Pi 5 PCIe interface
- expose hardware timestamping capability for PTP work
- expose PPS input and output functions through the NIC path
- support later NTP/PTP appliance behavior on the Pi 5
- allow careful experimentation with NIC-based PPS and PTP while preserving the existing ZED/FE/Teensy model

2.3 What TimeHAT is not allowed to change by default

Unless explicitly changed later, TimeHAT integration shall ****not****:

- replace ZED-F9T as GNSS truth
- replace FE-5680A as disciplined flywheel / holdover source
- replace Teensy as the main timing measurement engine
- reclassify Piksi as a truth source
- create ambiguous or mixed timing source presentation on the dashboard
- silently move the Pi from ZED-derived truth into an undefined NIC-derived truth model

3. Project objective

Integrate TimeHAT NIC hardware onto Raspberry Pi 5, after FE-5680A integration is complete, in a way that adds network hardware timestamping and PPS I/O capability for PTP/NTP experimentation and service delivery while preserving:

- ZED-F9T as the sole GNSS truth source
- ZED PPS as the primary second marker
- FE-5680A as the disciplined local oscillator and holdover source
- Teensy as the timing measurement and control engine
- Piksi as comparison-only PPS reference
- the database and dashboard as clear representations of the defined timing architecture

4. Scope

4.1 In scope

- Raspberry Pi 5 PCIe TimeHAT hardware integration
- TimeHAT NIC bring-up and Linux recognition
- hardware timestamping verification
- PPS input/output integration through the TimeHAT/i226 path
- system architecture definition for coexistence with FE-5680A and ZED
- telemetry, service, and dashboard implications of the new NIC timing path
- phased enablement for PTP and later NTP/PTP time appliance functions
- verification planning for NIC, PPS, and PTP behavior

4.2 Out of scope for initial phases

- changing GNSS truth away from ZED
- bypassing FE-5680A discipline/holdover design
- using TimeHAT as the sole timing truth without the existing architecture
- replacing Teensy with the i226 as the primary measurement engine
- redesigning the entire dashboard around TimeHAT-only timing semantics
- assuming stock kernel behavior is sufficient for every PPS I/O use case without verification

5. Guiding architecture principles

5.1 Truth hierarchy shall be preserved

The system shall preserve the following hierarchy:

1. ZED-F9T is the sole GNSS truth source
2. ZED PPS is the primary truth second marker
3. FE-5680A is the disciplined local flywheel / holdover source
4. Teensy is the primary timing measurement and control engine
5. TimeHAT is a network timing interface and hardware timestamping adjunct
6. Piksi remains comparison-only

5.2 TimeHAT shall be an interface layer, not a truth replacement

TimeHAT should be treated as:

- a PCIe NIC platform
- a PTP timestamping path
- a PPS I/O interface
- a service-delivery enabler for NTP/PTP

It should not be treated as an automatic replacement for the established truth model.

5.3 FE and TimeHAT roles shall remain distinct

- FE-5680A: stability, discipline, holdover
- TimeHAT/i226: network timestamping, PPS I/O, PTP client/server capabilities

5.4 Dashboard clarity shall be preserved

The dashboard shall continue to distinguish:

- source truth
- corrected timing state
- oscillator/holdover state
- PPS comparison state
- network/PTP interface state

6. Concept of operations

6.1 Normal operating mode after TimeHAT integration

In normal operation:

- ZED defines GNSS truth
- ZED PPS defines the primary second boundary
- FE-5680A remains the local disciplined flywheel
- Teensy measures key timing relationships
- Raspberry Pi continues to host chrony, database, dashboard, and time services
- TimeHAT provides NIC-based timestamping and network timing interfaces
- network clients may consume NTP and later PTP from the Pi 5 platform

6.2 NIC-assisted timing mode

In NIC-assisted timing mode:

- the TimeHAT i226 provides hardware timestamping support for PTP
- TimeHAT PPS input/output may be used where validated
- NIC timing services operate in a way bounded by the established truth hierarchy
- service roles are subordinate to the existing truth architecture rather than replacing it

6.3 Holdover-aware network service mode

If GNSS truth is lost:

- FE-5680A remains the flywheel source for local stability

- the system may continue advertising network time in degraded or holdover mode only if explicitly designed to do so
- degraded state shall be visible in telemetry and service state reporting
- TimeHAT shall not conceal loss of truth simply because the NIC remains active

7. Top-level project goals

Goal G1 - Preserve the existing truth model

TimeHAT integration must not blur or overwrite the current ZED/FE/Teensy truth model.

Goal G2 - Add robust NIC-based timestamping

The system should gain stable and testable i226 hardware timestamping capability on Raspberry Pi 5.

Goal G3 - Add validated PPS I/O through the TimeHAT path

The system should support tested and explicitly verified PPS input and/or output through TimeHAT where appropriate.

Goal G4 - Enable later PTP appliance capability

The system should support later deployment of PTP client/server or boundary-clock-like experimentation as appropriate to the architecture.

Goal G5 - Keep FE and network timing roles distinct

The system should keep oscillator discipline and holdover functions separate from NIC service and timestamping functions.

Goal G6 - Keep observability strong

TimeHAT state, PTP state, timestamping health, and PPS path status should be visible in telemetry and dashboard outputs.

Goal G7 - Degrade transparently

If TimeHAT PPS/PTP paths fail or become invalid, the system should fail safe without damaging the baseline ZED/FE timing architecture.

8. Functional requirements

8.1 Architecture requirements

FR-A1

The system shall preserve ZED-F9T as the sole GNSS timing truth source.

FR-A2

The system shall preserve ZED PPS as the primary PPS reference unless an explicit later architecture change is approved.

FR-A3

The system shall preserve FE-5680A as the disciplined local oscillator and holdover source after FE integration is completed.

FR-A4

The system shall preserve Teensy as the primary timing measurement and control engine.

FR-A5

The system shall treat TimeHAT as a NIC, PPS I/O, and hardware timestamping interface layer rather than as the default replacement truth source.

FR-A6

The system shall preserve Piksi as a comparison-only PPS reference.

8.2 Hardware integration requirements

FR-HW1

The Raspberry Pi 5 shall mechanically and electrically support the TimeHAT PCIe connection in a stable and serviceable configuration.

FR-HW2

The system shall verify that the TimeHAT i226 NIC enumerates correctly on the Raspberry Pi 5 PCIe bus.

FR-HW3

The system shall verify power, thermal, and link stability for the TimeHAT NIC under expected operating conditions.

FR-HW4

The system shall define PPS input and PPS output wiring in a way that does not disturb the existing ZED and FE timing paths.

FR-HW5

Any TimeHAT PPS input path shall be validated for signal polarity, level compatibility, and timestamping correctness before being used operationally.

FR-HW6

Any TimeHAT PPS output path shall be validated for top-of-second alignment and service correctness before being relied upon.

FR-HW7

The system should support maintaining clean cable routing and grounding so TimeHAT timing I/O does not inject avoidable noise or confusion into existing timing paths.

8.3 Linux and driver requirements

FR-L1

The system shall identify the active kernel and driver path used by the TimeHAT i226 interface before enabling advanced PPS/PTP functions.

FR-L2

The system shall verify whether the stock igc driver is sufficient for the desired PPS I/O behavior.

FR-L3

If stock driver behavior is inadequate, the system shall support a controlled and documented patched-driver workflow.

FR-L4

The system shall keep driver modifications traceable to a specific kernel version and reproducible installation path.

FR-L5

The system should support rollback to a known-good network driver state.

FR-L6

The system shall not assume PPS input/output correctness merely because the NIC link is up.

8.4 Measurement and timestamping requirements

FR-M1

The system shall verify hardware timestamping capability of the TimeHAT i226 NIC.

FR-M2

The system shall verify that NIC timestamping services can run without disturbing the existing ZED/FE/Teensy pipeline.

FR-M3

The system shall measure and log whether PPS input timestamps are plausible and top-of-second aligned where expected.

FR-M4

The system shall measure and log whether PPS output is plausibly aligned to the intended timescale.

FR-M5

The system should compare NIC-timestamped timing behavior against the existing Teensy/ZED reference chain before elevating TimeHAT timing functions operationally.

FR-M6

The system shall distinguish between “NIC link healthy” and “timing path healthy”.

8.5 PTP and network timing service requirements

FR-P1

The system shall support staged PTP capability enablement rather than immediate full operational reliance.

FR-P2

The system shall support verification of ptp4l/phc2sys or equivalent tooling against the TimeHAT i226 hardware timestamping path.

FR-P3

The system shall make the active timing source for network service explicit.

FR-P4

The system shall not advertise a higher confidence timing state to network clients than the system can justify from current source validity.

FR-P5

The system should support later PTP server or grandmaster-like experimentation only after the NIC timing path has been fully validated.

FR-P6

Any network time service state exposed by the system shall distinguish among tracking, holdover, degraded, and fault conditions.

8.6 PPS interface requirements

FR-PP1

The system shall define which signal may feed TimeHAT PPS input during each project phase.

FR-PP2

The preferred initial TimeHAT PPS input truth source shall be derived from the existing truth hierarchy rather than from Piksi.

FR-PP3

The system shall not reroute the core ZED PPS path in a way that jeopardizes chrony or Teensy baseline operation during early TimeHAT phases.

FR-PP4

The system shall validate edge polarity, timestamp direction, and alignment before TimeHAT PPS input is considered usable.

FR-PP5

The system shall validate TimeHAT PPS output against an external reference before using it as an appliance output.

FR-PP6

The system should support clear documentation of when TimeHAT PPS is in monitor-only mode versus service-delivery mode.

8.7 Telemetry, database, and dashboard requirements

FR-T1

The system shall expose TimeHAT integration state separately from ZED, FE, and Piksi state.

FR-T2

The database and/or API layer shall distinguish TimeHAT NIC health, PTP health, PPS I/O health, and driver state.

FR-T3

The dashboard shall present TimeHAT as a network timing interface section rather than as the GNSS truth section.

FR-T4

The dashboard shall preserve existing top timing summary semantics for corrected timing, oscillator state, and PPS comparison.

FR-T5

The dashboard should present fields such as:

- TimeHAT detected
- i226 link state
- hardware timestamping state
- PTP service state
- PPS input valid
- PPS output valid
- active driver mode
- degraded/holdover advertisement state

FR-T6

The system shall sanitize missing or invalid NIC/PTP/PPS values before visualization.

8.8 Safety and robustness requirements

FR-S1

The system shall preserve current ZED/FE/Teensy timing operation if the TimeHAT subsystem is offline or faulty.

FR-S2

The system shall fail safe if TimeHAT PPS input is invalid, ambiguous, or driver-broken.

FR-S3

The system shall fail safe if TimeHAT PTP service becomes unstable or misleading.

FR-S4

The system shall log or expose driver, timestamping, and PPS faults for troubleshooting.

FR-S5

The system shall distinguish between:

- NIC absent
- NIC present but untuned
- timestamping available
- PPS monitor-only
- validated service mode
- degraded
- fault

9. Non-functional requirements

NFR-1 Stability

TimeHAT integration should not reduce baseline timing stability.

NFR-2 Modularity

TimeHAT should be addable and removable without forcing redesign of the FE/ZED/Teensy core timing architecture.

NFR-3 Observability

Major TimeHAT states should be visible through telemetry, logs, or dashboard outputs.

NFR-4 Maintainability

Driver, kernel, service, and configuration changes should be documented and reproducible.

NFR-5 Traceability

Each significant TimeHAT function should map to identifiable hardware paths, services, telemetry, and validation tests.

NFR-6 Defensive service design

The system should avoid advertising misleading timing confidence over the network.

10. TimeHAT subsystem state model

Recommended logical states:

- `ABSENT`
- `DETECTED`
- `LINK\_OK`
- `TIMESTAMPING\_OK`
- `PPS\_MONITOR\_ONLY`
- `PPS\_VALIDATED`
- `PTP\_TESTING`
- `PTP\_SERVICE`
- `DEGRADED`
- `FAULT`

State meanings

- `ABSENT`: hardware not detected
- `DETECTED`: NIC enumerated but not yet timing-validated
- `LINK\_OK`: network link functional
- `TIMESTAMPING\_OK`: hardware timestamping confirmed
- `PPS\_MONITOR\_ONLY`: PPS path connected and under test but not trusted for service
- `PPS\_VALIDATED`: PPS timing behavior validated
- `PTP\_TESTING`: active PTP experiments underway
- `PTP\_SERVICE`: network time service enabled and validated
- `DEGRADED`: timing service operating with reduced confidence, such as holdover-aware mode
- `FAULT`: invalid or misleading condition detected

11. Initial telemetry and field plan

Recommended fields:

- ``timehat_present``
- ``timehat_state``
- ``timehat_driver``
- ``timehat_kernel_version``
- ``timehat_link_up``
- ``timehat_speed_mbps``
- ``timehat_hwtstamp_valid``
- ``timehat_pps_in_valid``
- ``timehat_pps_out_valid``
- ``timehat_ptp_service_state``
- ``timehat_phc_offset_ns``
- ``timehat_phc_lock_valid``
- ``timehat_degraded``
- ``timehat_fault_reason``

Optional later fields:

- ``timehat_phc_freq_ppb``
- ``timehat_pps_in_edge_mode``
- ``timehat_pps_out_align_valid``
- ``timehat_driver_patch_state``
- ``timehat_advertised_timescale``
- ``timehat_client_count``

These fields should remain clearly separated from:

- ZED truth fields
- FE holdover fields
- Teensy timing summary fields
- Piksi comparison fields

12. Dashboard display plan

12.1 Existing panels to preserve

The dashboard should continue to preserve:

- corrected timing summary
- auto-cal / phase bias interpretation
- GNSS health driven from ZED-backed DB values
- Piksi-ZED comparison
- FE oscillator / holdover section once FE is complete

12.2 New TimeHAT section

A dedicated TimeHAT / Network Timing section should include:

Cards

- TimeHAT state
- i226 link state
- driver mode
- hardware timestamping state
- PPS input validity
- PPS output validity
- PTP service state
- degraded / holdover advertisement state

Charts or diagnostics

- PHC offset trend if implemented
- PTP lock trend if implemented
- PPS validation trend if implemented

Narrative analysis rules

The deterministic narrative panel may later summarize:

- TimeHAT absent
- NIC present but timing not validated
- PPS monitor-only mode

- timestamping healthy
- PTP testing active
- service mode active
- degraded advertisement state
- fault or driver mismatch

13. Interface requirements

13.1 Hardware interfaces

The following shall be documented:

- Pi 5 PCIe/FPC connection to TimeHAT
- TimeHAT NIC Ethernet port usage
- TimeHAT PPS input source during each phase
- TimeHAT PPS output destination during each phase
- coexistence rules with ZED PPS, FE, Teensy, and any external measurement gear

13.2 Software interfaces

The following shall be documented:

- kernel/driver requirements
- PTP tools and service files
- interaction with chrony and/or phc2sys
- dashboard/API exposure of TimeHAT state
- any driver patch workflow and rollback procedure

14. Phased implementation strategy

Phase 0 - Preserve completed FE baseline

Objective

Freeze and verify the post-FE baseline before adding TimeHAT.

Hardware tasks

- leave ZED, FE, Teensy, and Piksi timing paths unchanged
- do not reroute PPS for TimeHAT yet

Software tasks

- verify existing FE-complete baseline health
- refresh snapshot
- commit known-good state

Exit criteria

- stable post-FE baseline recorded

Phase 1 - Mechanical and NIC bring-up

Objective

Physically install TimeHAT on Pi 5 and verify basic NIC function.

Hardware tasks

- mount TimeHAT and PCIe/FPC connection cleanly
- verify power and thermal behavior
- connect Ethernet to test switch

Software tasks

- verify PCIe enumeration
- verify NIC link
- record kernel and driver versions
- confirm no regressions in existing timing stack

Exit criteria

- i226 NIC detected and link stable

Phase 2 - Hardware timestamping verification

Objective

Verify the TimeHAT NIC can provide usable hardware timestamping.

Hardware tasks

- none beyond stable mounted operation

Software tasks

- inspect hardware timestamping capabilities
- verify PHC exposure
- run basic PTP test tooling
- log whether timestamping is available and sane

Exit criteria

- timestamping confirmed without disturbing baseline time server operation

Phase 3 - Driver and PPS capability validation

Objective

Determine whether stock driver behavior is sufficient and validate PPS I/O support.

Hardware tasks

- connect a known reference PPS path to the intended TimeHAT input only if safe

- keep baseline ZED/Teensy/Pi truth path intact

Software tasks

- test PPS input behavior
- test PPS output behavior
- determine if stock driver works acceptably
- if needed, evaluate patched igc path in a controlled manner
- document installation and rollback path

Exit criteria

- known-good PPS capability path identified and documented

Phase 4 - Monitor-only PPS integration

Objective

Add TimeHAT PPS in monitor-only mode without making it authoritative.

Hardware tasks

- feed a validated truth-derived PPS signal into TimeHAT input for observation only
- do not disturb core chrony and Teensy paths

Software tasks

- log PPS input validity
- compare TimeHAT PPS timing behavior against existing reference chain
- expose monitor-only state in telemetry and dashboard

Exit criteria

- TimeHAT PPS input can be observed reliably without altering truth architecture

Phase 5 - TimeHAT telemetry and dashboard integration

Objective

Expose NIC timing state to the software stack.

Hardware tasks

- none

Software tasks

- add TimeHAT fields to DB/API layer as appropriate
- add dedicated dashboard section
- preserve existing summary semantics
- add defensive rendering for absent/invalid values

Exit criteria

- TimeHAT status visible and clearly separated from GNSS/FE/Teensy truth panels

Phase 6 - PTP lab validation

Objective

Test PTP behavior using the TimeHAT hardware timestamping path.

Hardware tasks

- connect appropriate test network peers and measurement setup

Software tasks

- run ptp4l/phc2sys or equivalent
- characterize offsets and stability
- determine whether the Pi can serve as a useful PTP endpoint using the current architecture
- document whether service mode should be allowed

Exit criteria

- PTP behavior characterized and bounded

Phase 7 - Controlled service mode

Objective

Enable network timing service in a controlled, truthful way.

Hardware tasks

- none beyond validated operational configuration

Software tasks

- define when TimeHAT-backed PTP or related services may be active
- make advertised timing source explicit
- expose degraded and holdover-aware states
- ensure service confidence tracks source validity

Exit criteria

- network timing service can run without misrepresenting source confidence

Phase 8 - Holdover-aware network behavior

Objective

Define network behavior when GNSS truth is lost but FE holdover remains available.

Hardware tasks

- none

Software tasks

- specify what service state should be exported during holdover
- define degraded service indicators
- ensure TimeHAT does not hide loss of GNSS truth behind an apparently healthy NIC state

Exit criteria

- holdover-aware network service behavior is deterministic and visible

15. Verification strategy

15.1 Baseline verification

Before TimeHAT integration:

- verify completed FE baseline
- verify chrony and dashboard health
- verify Teensy telemetry stability

15.2 NIC verification

After physical install:

- verify PCIe detection
- verify link stability
- verify thermal sanity

15.3 Timestamping verification

- verify PHC availability
- verify hardware timestamping configuration
- verify timestamping plausibility

15.4 PPS verification

- verify PPS input correctness
- verify PPS output alignment
- compare against external reference and existing timing chain

15.5 PTP verification

- verify ptp4l/phc2sys behavior
- verify stable state under expected link conditions
- verify service state matches true timing source confidence

15.6 Fault/degraded verification

- verify driver failure handling
- verify absent-NIC handling
- verify degraded holdover-aware advertisement rules

16. File and workflow implications

Potential files or areas likely to be touched later:

- chrony configuration if service interaction changes
- PTP-related service/unit files
- dashboard backend and frontend files
- collector/API logic for TimeHAT state exposure
- snapshot and diagnostic bundle scripts
- driver installation or rollback notes stored in repo documentation

Workflow rules:

7. confirm live runtime paths first
8. edit live copy if runtime change is needed
9. sync live changes back to repo copies
10. refresh snapshot
11. commit and push

17. Risks and design cautions

R1

Do not let NIC presence be mistaken for timing truth.

R2

Do not let TimeHAT dashboard presentation blur ZED truth versus service interface role.

R3

Do not trust PPS I/O behavior without explicit validation, especially if driver patching is involved.

R4

Do not allow TimeHAT experiments to disturb the stable FE/ZED/Teensy architecture during early phases.

R5

Do not advertise network timing confidence that exceeds current truth validity.

18. Summary

This specification defines a safe integration path for TimeHAT NIC on Raspberry Pi 5 after FE-5680A integration is complete.

The central design intent is:

****ZED remains truth, FE remains the disciplined flywheel, Teensy remains the measurement engine, and TimeHAT extends the system with NIC-based timestamping, PPS I/O, and later PTP-capable service functions without collapsing the established timing architecture.****